

Why tools matter

LESSON 16 · MODULE 4 — TOOLS & WORKFLOWS

What you'll learn

- Why an agent is a **text model** that, by default, can't query your database, hit an API, or check live state
- How `agent.add_tool()` hands the agent a plain Python function and lets it decide when to call it
- How the **docstring** becomes the **description** and **type hints** become the **schema** — no decorators, no special class
- The picking rule for tools vs. RAG vs. prompt context

Key concepts

****An agent is a text model — it can reason, but it can't *do*.**** Ask about a specific product SKU and a tool-less agent will admit it can't see real-time inventory and offer to interpret a snapshot if you paste one. Honest, but the customer still has no answer. The agent can reason about the question and tell you which fields to pull; it just can't go look at the data itself. That's the gap tools close.

A tool is just a Python function. No decorators, no special class — anything callable. The **docstring** becomes the description the model reads to decide *when* to call it; the **type hints** become the schema that tells it *what* to pass. The SDK handles the call-and-return loop. In a real system the function hits a database or an API; for a demo a hard-coded dict has the same shape from the agent's perspective.

TAKEAWAY

A tool is just a Python function.

`agent.add_tool()` wires it up in one line. Register the function, rerun the same question, and the agent reads the docstring, recognizes the tool fits, parses the argument out of your prompt (e.g. the SKU `GH-2400`), calls the function, gets the result, and reformats it into a clean answer. Docstring tells the model what the tool does; type hints tell it what to pass; the SDK handles the call.

Tools, RAG, and prompt context solve different problems. Tools are for *actions and real-time state* — anything the agent needs to do or look up that isn't in its training data and isn't a static document: inventory lookups, DB queries, API calls, sending email, calculations on live data. If the answer changes after training, you need a tool. **RAG** is for *static knowledge* you control — docs, policies, manuals, read once per query (videos 10–15). **Prompt context** is for *small, stable* info — the user's name, the date, system rules — a few hundred tokens that don't justify infrastructure.

TAKEAWAY

Tools, RAG, and prompt context solve different problems.

The picking rule is short. Tools when the answer requires *doing* something. RAG when the answer is a *document* you control. Prompt when it's a *sentence or two* of context. Most production agents use all three.

TAKEAWAY

The picking rule is short.

Code shown in this lesson

PYTHON

```
# One install tweak for notebooks: the [jupyter] extra pulls in nest_asyncio so tool
# calls work inside a notebook event loop.
# pip install "lyzr-adk[jupyter]"
```

PYTHON

```
# Tool-less agent – honest, but stuck.
agent = studio.create_agent(
    name="support", provider="gpt-4o",
    role="customer support agent",
    goal="answer product and inventory questions",
)
agent.run("Is SKU GH-2400 in stock?")
# → "I can't see real-time inventory, but if you paste a snapshot I can interpret it."
```

PYTHON

```
# A tool is just a Python function. Docstring = description, type hints = schema.
def check_inventory(sku: str) -> str:
    """Look up live inventory status for a product SKU."""
    # Real system: hit a database or API. Demo: a hard-coded dict, same shape.
    inventory = {"GH-2400": "in stock, 47 units, DC East"}
    return inventory.get(sku, "SKU not found")
```

PYTHON

```
# Hand the function to the agent, rerun the same question.
agent.add_tool(check_inventory)
agent.run("Is SKU GH-2400 in stock?")
# → "In stock – 47 units (DC East)." The agent parsed GH-2400, called the tool, reformatted.
```

TRY THIS

- 1 Run the question once *before* `add_tool` and once *after*. Diff the answers — the before is an honest dodge, the after is a real answer.
- 2 Change `check_inventory`'s docstring to be vague ("does stuff with a sku") and see whether the agent still reliably picks it. Restore a crisp docstring and watch reliability return — proof the docstring is the interface.
- 3 Add a second tool (e.g. `get_price(sku: str) -> float`) and ask a question that needs both. Watch the agent chain the calls and combine the results.